
Client/Institute Name	:	KIOT – SDET 2026
Course	:	Automation Testing with Selenium - Python
Technical Assessment	:	02
Module	:	Pytest with Page Object Model, Reports and Parallel Execution
Date	:	17-Jun-26
Time	:	02:00 P.M – 05:00 P.M
Duration	:	3 h
Total Marks	:	50 Marks

Part A – 1 x 10 = 10 Marks**Part A – MCQ (10 Marks)****Part B — Practical Implementation (40 Marks)****Question 1: E-Commerce Website Automation using Selenium WebDriver (20 Marks)****Application URL:** <https://practice.automationtesting.in/shop/>**Background of the Problem Statement**

The organization has decided to automate select functionalities of its E-Commerce website to strengthen its regression testing efforts. As an SDET, you are required to design and implement Selenium WebDriver test scripts that satisfy the following test cases and requirements.

Test Case 01: Validate the Price of the HTML5 Forms Book

1. Launch the browser and navigate to <https://practice.automationtesting.in/shop/>
2. Locate the product titled “HTML5 Forms” on the shop listing page.
3. Retrieve and validate the displayed price of the book.

Expected Result: *The price of the “HTML5 Forms” book is displayed as ₹280.00.***Test Case 02: Validate Navigation to the JS Data Structures and Algorithm Book**

1. Navigate to <https://practice.automationtesting.in/shop/>
2. Click on the product “JS Data Structures and Algorithm” from the shop listing.
3. Validate the product detail page that opens.

Expected Result: *Only the “JS Data Structures and Algorithm” product detail page is opened, with the correct product title and details, and no other book is displayed.***Test Case 03: Validate the Book Count under the HTML Category**

1. Navigate to <https://practice.automationtesting.in/shop/>
2. Click on the category filter “HTML (3)” from the category list.
3. Count the number of books displayed in the filtered results.

Expected Result: The number of books displayed after filtering equals the count specified in the category label, i.e., 3.

Test Case 04: Validate Sorting by Price (Low to High)

1. Navigate to <https://practice.automationtesting.in/shop/>
2. From the sorting dropdown, select the option "Sort by Price: Low to High."
3. Capture the list of displayed prices after sorting and validate the order.

Expected Result: The books are rearranged and displayed in ascending order of price.

Test Case 05: Validate Presence of the Subscribe Button

1. Navigate to <https://practice.automationtesting.in/shop/>
2. Use JavascriptExecutor to scroll to the bottom of the page.
3. Validate that the Subscribe button/section is present and visible.

Expected Result: The Subscribe button is present and visible at the bottom of the page.

Requirements to be Met

1. Group Test Case 01 and Test Case 02 together as a Smoke Test Suite.
2. Group Test Case 03 and Test Case 04 together as a Regression Test Suite.
3. Use JavascriptExecutor for at least one applicable action in the scripts (for example, scrolling to an element).
4. Every test case must include the appropriate Assert method(s) (e.g., assertEquals, assertTrue) to validate the expected result.

Marks Distribution

Test Case / Requirement	Marks
Test Case 01 – HTML5 Forms Book Price Validation	3
Test Case 02 – JS Data Structures and Algorithm Navigation	3
Test Case 03 – HTML Category Book Count Validation	4
Test Case 04 – Sort by Price (Low to High) Validation	4
Test Case 05 – Subscribe Button Validation (JavascriptExecutor)	3
Compliance with Requirements (Grouping, JavascriptExecutor, Assertions)	3
Total	20

Question 2: E-Commerce Automation using PyTest and Page Object Model (20 Marks)

Application URL: <https://tutorialsninja.com/demo>

Background of the Problem Statement

To strengthen its automation framework further, the organization intends to adopt the PyTest framework along with the Page Object Model (POM) design pattern. As an SDET, you are required to design a Python-based Selenium automation framework for the Tutorials Ninja Demo e-commerce application that satisfies the following test cases and requirements, while demonstrating the PyTest and POM concepts listed below.

Concepts Covered

Concept Area	Topics Covered
PyTest	Running test methods, Assertions, Flags, User-defined & Custom Markers, Skipping Tests, Grouping Tests, Parameterization, Hook Functions, Individual & Parallel Execution
Page Object Model	Introduction to POM, Page Factory, POM Optimization with Reusable Methods

Design test scripts and framework components for the following requirements:

Test Case 01: Validate New User Registration

1. Launch the browser and navigate to <https://tutorialsninja.com/demo>
2. Navigate to the Register page from the “My Account” menu.
3. Fill in the registration form with valid, dynamically generated user details and submit it.

Expected Result: A confirmation message, “Your Account Has Been Created!”, is displayed.

Test Case 02: Validate Login with Multiple Data Sets

1. Navigate to the Login page.
2. Using PyTest parameterization, execute the login flow with at least three data sets: valid credentials, an invalid email format, and an incorrect password.
3. Capture and validate the resulting page or message for each data set.

Expected Result: Valid credentials redirect to the “My Account” page; each invalid data set displays the corresponding error message.

Test Case 03: Validate Product Search Functionality

1. On the home page, search for a product (for example, “MacBook”) using the search box and submit the search.
2. Capture the list of products displayed on the search results page.
3. Validate that the results contain the expected product.

Expected Result: The search results page displays at least one product containing the searched keyword.

Test Case 04: Validate Add to Cart Functionality

1. Navigate to a product page and add the product to the cart.
2. Use a PyTest hook function (via `conftest.py`) to handle browser setup before the test and teardown after the test.
3. Validate that the cart count and cart total update correctly after the product is added.

Expected Result: The cart icon/count reflects the added item, and the cart total matches the expected amount.

Requirements to be Met

1. Design Page classes following the Page Object Model, initializing web elements using the Page Factory approach, for each page involved (Home Page, Registration Page, Login Page, Search Results Page, Product Page, Cart Page).
2. Optimize the POM design by implementing reusable/common methods (for example, navigation, explicit waits, element interaction helpers) shared across Page classes.
3. Group Test Case 01 and Test Case 03 under a Smoke Test Suite, and Test Case 02 and Test Case 04 under a Regression Test Suite, using custom PyTest markers.
4. Implement at least one Hook function (via conftest.py) to manage setup/teardown shared across the test cases.
5. Demonstrate execution of the test suite individually (a single test case via the command line) as well as in parallel (using the pytest-xdist plugin), specifying the command-line flags used.
6. Every test case must include the appropriate Assertion(s) to validate its expected outcome.

Marks Distribution

Test Case / Requirement	Marks
Test Case 01 – New User Registration (Smoke)	4
Test Case 02 – Parameterized Login Validation (Regression)	4
Test Case 03 – Product Search Validation (Smoke)	4
Test Case 04 – Add to Cart with Hook-based Setup/Teardown (Regression)	4
Compliance with Requirements (POM, Reusable Methods, Markers, Parallel Execution)	4
Total	20